Technical Report: AdaptiveRoute

Agentic Route Replanning with a Fine-Tuned Policy and an Exact Solver

Gabriel Carvalho Domingos da Conceição

August 29, 2026

Abstract

This report documents the design, implementation and evaluation of AdaptiveRoute, a system for replanning last-mile delivery routes when operational disruptions occur mid-execution. The system converts a driver's natural-language report — a blocked road, an unavailable customer — into a validated route plan. It combines three components with strictly separated responsibilities: a general-purpose language model that interprets the message and orchestrates the workflow, a fine-tuned 7B policy that proposes route candidates, and an exact Pyomo + HiGHS optimizer that remains the sole authority on feasibility. No plan reaches a driver without passing deterministic validation, regardless of which component produced it. Around this core the system carries the supporting AI-engineering layer that makes a conversational interface viable: a bounded rolling context window, a vector-backed retrieval layer over project documentation, and a grounded route question-answering pipeline. The report covers the architecture, the agentic workflow, the validation contract, these components, the supervised fine-tuning of the routing policy, and the measured limits of the resulting system.

Contents

1 Executive Summary

AdaptiveRoute addresses a gap that is neither an optimization problem nor a language problem, but the handoff between them. A delivery route optimized overnight becomes invalid during execution; re-running the optimizer requires someone to translate a disruption into a formal model, and asking a language model directly produces plausible routes that violate capacity or skip customers.

The system closes that handoff with a three-layer architecture:

- a general-purpose LLM interprets the driver's message and extracts a structured operational event;

- a fine-tuned LoRA policy proposes a route candidate in approximately 1.7 seconds;
- deterministic validation accepts, repairs, or rejects that candidate, with Pyomo + HiGHS as the fallback authority.

The fine-tuned routing policy, referred to as v5, reaches a 94.4% feasible-candidate rate on a fixed 1,000-example benchmark. The remaining 5.6% is not a correctness risk, because the architecture never depends on the model being right: invalid candidates are repaired or replaced by the solver. The measured operating boundary of the policy is 8–9 customers per scenario; beyond that the cascade falls through to the solver on every request.

The selected adapter is `outputs/models/adaptiveroute-qwen2_5-7b-lora-error20k-v5`, based on `Qwen/Qwen2.5-7B-Instruct`.

2 Project Objective and Scope

The objective was to build a working end-to-end system, not a demonstration of isolated components. The scope covers:

- exact CVRP planning over ingested orders;
- natural-language reporting of operational disruptions;
- agentic replanning with validation, repair and fallback;
- conversation memory and route question answering;
- an operational console for administrators and a scoped workspace for drivers.

The routing scenarios are synthetic CVRP instances, generated primarily with 8 customers and 2 vehicles. The target behaviour of the learned component is operational feasibility, not proof of optimality: a generated plan is useful only if it passes deterministic validation.

3 Theoretical Foundation

3.1 Vehicle Routing and CVRP

The Vehicle Routing Problem was introduced by Dantzig and Ramser in the classical truck dispatching formulation [1]. In the Capacitated Vehicle Routing Problem, a fleet of vehicles with limited capacity must serve a set of customers from a depot while minimizing total travel distance.

CVRP is NP-hard. For realistic instances, exact optimization becomes expensive, and practical systems combine mathematical programming, heuristics and learned policies.

3.2 Prescriptive Optimization and Subtour Elimination

The prescriptive layer answers: given a scenario and a disruption, what route plan should be executed? Pyomo was selected because it allows explicit algebraic modeling in Python and keeps the optimization model inspectable, testable and version-controlled [10]. HiGHS was selected as the open-source solver for LP and MIP problems, with a Python interface suitable for local experimentation [11].

Subtour elimination uses the Miller–Tucker–Zemlin formulation [2]. MTZ is compact and readable, requiring only $O(n^2)$ additional constraints, which suits a system where the model is meant to be auditable. Its known weakness is a loose linear relaxation compared with Dantzig–Fulkerson–Johnson cut families, which becomes the binding constraint on scalability as instance size grows. This trade-off is revisited in Section 14.

3.3 Learning for Combinatorial Optimization

A substantial literature asks whether neural networks can solve routing problems directly. Pointer Networks introduced an attention-based architecture able to output permutations over variable-length inputs, and demonstrated it on TSP [3]. Kool, van Hoof and Welling extended this line with an attention model trained by reinforcement learning that produces high-quality tours for TSP and VRP without a solver in the loop [4].

These approaches replace the optimizer. They achieve competitive tour lengths but provide no feasibility guarantee, which is disqualifying when a violated capacity constraint means a vehicle that physically cannot carry its load.

Bengio, Lodi and Prouvost survey the alternative framing, in which machine learning assists rather than replaces exact methods — learning to guide branching, to select heuristics, or to produce warm starts [5]. AdaptiveRoute adopts that hybrid stance and applies it at a different level: the learned component is a fine-tuned language model operating on a natural-language interface, and the exact solver retains final authority over every plan. The contribution is not a better tour-construction network; it is an architecture in which an unreliable fast component can be used safely.

3.4 Transformer Language Models

The base model is a causal Transformer [6]. The attention mechanism suits this task because route decisions must be conditioned on relationships among customers, vehicles, capacity constraints, blocked arcs and the previous plan. An instruction-tuned model was preferred because the task is contract-driven: the model must follow a structured prompt and emit valid JSON.

3.5 LoRA and QLoRA

Full fine-tuning of a 7B-parameter model was not practical on the available hardware. The project used parameter-efficient fine-tuning with LoRA, which freezes the pretrained weights and injects trainable low-rank matrices into selected layers [7]. Training also used QLoRA-style 4-bit loading, keeping the base model quantized with NF4 and double quantization while adapter weights are updated [8]. The base `Qwen/Qwen2.5-7B-Instruct` model remained frozen throughout.

4 System Architecture

4.1 Component Overview

```
Admin / Driver browser
        |
        v
React frontend  (admin console, driver workspace, Leaflet maps)
        |
        v
FastAPI backend
        |
        +--> scenario, driver, route and planning APIs
        +--> agentic routing API
        +--> conversation memory API
        +--> RAG API
        +--> map geometry API
        |
        +--> MongoDB          conversations, messages, context windows,
```

```
|                       agent runs, drivers, routes, planning jobs
+--> PostgreSQL         document chunks and embeddings (pgvector)
+--> Pyomo + HiGHS      deterministic CVRP solve, fallback authority
+--> OSRM               road distance matrix and route geometry
+--> OpenAI-compatible LLM API
```

Three interfaces are declared as Python protocols and are swappable at runtime by environment variable: the routing engine, the route-candidate generator and the event extractor. The repository layer provides in-memory implementations, so the full test suite runs without a GPU, a database or a model server.

4.2 Model Responsibilities

The system runs two language models with disjoint responsibilities. This separation is deliberate and is one of the central design decisions.

Capability	Model or engine	Role
Exact planning	Pyomo + HiGHS	Prescriptive source of truth.
Interaction and orchestration	Qwen2.5-Coder-32B	Event extraction, route Q&A, explanation.
Route candidate	Qwen2.5-7B + LoRA v5	Fast approximate replan under disruption.
Guardrails	Deterministic validator	Feasibility authority.

The rationale is right-sizing. Interpreting an ambiguous driver message is an open-ended language task that benefits from breadth, so it is handled by a large general model used off the shelf, at temperature 0.1. Emitting a valid route sequence is a narrow, structured task that benefits from precision and latency, so it is handled by a small fine-tuned specialist at temperature 0. The specialist sits on the hot path, which means 32B inference is not paid for every candidate.

The two are configured independently through separate base URLs, model names and temperatures (`ADAPTIVEROUTE_ORCHESTRATOR_*` and `ADAPTIVEROUTE_ROUTING_POLICY_*`). The 32B model never produces a route; the 7B model never addresses a driver.

4.3 Storage and Deployment

MongoDB stores document-shaped operational state: conversations, messages, rolling context windows, agent traces, drivers, operational routes and planning jobs. PostgreSQL with pgvector stores retrieval chunks and embeddings for documentation-based RAG.

The Docker stack contains the FastAPI backend, the React frontend, MongoDB, PostgreSQL, an optional OSRM service for road distances and geometry [12], and an optional GPU profile that loads the LoRA adapter directly in the API process. The OpenAI-compatible LLM API runs outside the Compose stack and is reached through `host.docker.internal`.

5 Prescriptive Optimization Component

The Pyomo + HiGHS engine has three roles: it generates supervised labels for training, it provides the fallback plan when a candidate cannot be accepted, and it is the reference against which the learned policy is measured.

Let N be the node set, $C \subset N$ the customers, K the vehicles, d_{ij} the travel distance on arc (i, j), q_j the demand of customer j, Q_k the capacity of vehicle k, and B the set of blocked arcs. Let $x_{ijk} \in \{0, 1\}$ indicate that vehicle k traverses arc (i, j), and $y_{jk} \in \{0, 1\}$ that vehicle k serves customer j.

The objective minimizes total distance:

$$\min \sum_{k\in K}\sum_{i\in N}\sum_{j\in N, j\neq i} d_{ij}\, x_{ijk}$$

Each customer is served exactly once:

$$\sum_{k\in K} y_{jk} = 1 \quad \forall j \in C$$

Assignment is linked to routing through degree constraints, so that a vehicle enters and leaves a customer if and only if it serves that customer:

$$\sum_{i\in N, i\neq j} x_{ijk} = y_{jk}, \qquad \sum_{i\in N, i\neq j} x_{jik} = y_{jk} \quad \forall j \in C, \forall k \in K$$

Capacity is then enforced on the assignment variables:

$$\sum_{j\in C} q_j\, y_{jk} \leq Q_k \quad \forall k \in K$$

Blocked arcs are excluded:

$$x_{ijk} = 0 \quad \forall (i,j) \in B, \forall k \in K$$

Subtours are eliminated with MTZ ordering variables u_{jk}:

$$u_{ik} - u_{jk} + |C|\, x_{ijk} \leq |C| - 1 \quad \forall i,j \in C,\ i \neq j,\ \forall k \in K$$

The implementation adds depot start and end rules and vehicle-usage linking. The linking constraints above matter: without them, capacity would be decoupled from the routing variables and could be satisfied vacuously.

6 Agentic Replanning Workflow

6.1 Graph

The workflow is implemented with LangGraph. Each node is a specialized agent class inheriting from `RoutingWorkflowAgent`, which keeps the graph wiring thin and gives every step a shared contract for trace and error handling.

```
extract_event
      |
      v
solve_base
      |
      v
apply_event
      |
      v
generate_candidate
      |
      v
validate_candidate
      |
      +-- valid ----> compose_response
      |
```

```
+-- invalid --> repair_candidate
                    |
                    +-- valid ----> compose_response
                    |
                    +-- invalid --> solver_fallback
                                         |
                                         v
                                  compose_response
```

If event extraction finds no new event but a context window exists, the workflow answers as a conversational follow-up rather than forcing a replanning action.

6.2 Agent Responsibilities

Agent	Responsibility
`ExtractEventAgent`	Extract the operational event from user text; report method and confidence.
`SolveBaseAgent`	Solve the current scenario before mutation; validate the base plan.
`ApplyEventAgent`	Apply the disruption to the scenario and record the mutation diff.
`GenerateCandidateAgent`	Generate a route candidate from the solver, an API model or a local LoRA.
`ValidateCandidateAgent`	Check candidate feasibility against the deterministic contract.
`RepairCandidateAgent`	Attempt conservative structural repair of an invalid candidate.
`SolverFallbackAgent`	Re-solve with Pyomo + HiGHS when candidate and repair both fail.
`ComposeResponseAgent`	Build the response payload with plan, validation, comparison and trace.

6.3 Event Extraction

Supported events are blocked arc, customer unavailable and priority change. Extraction runs in two modes: a deterministic rule-based extractor, and an LLM extractor backed by the orchestrator model with rule-based fallback.

The LLM extractor is not trusted blindly. Its output is parsed into an explicit event object and rejected if it references nodes that do not exist in the scenario or an unsupported event type. If the provider is unavailable or returns invalid JSON, extraction falls back to the rule-based path.

6.4 Candidate Generation Modes

Backend	Behaviour
`solver`	Pyomo + HiGHS as candidate generator. Deterministic; used in tests.
`api`	The trained LoRA served through an OpenAI-compatible API.
`local`	The LoRA adapter loaded directly in the API process.

Both model-backed modes first check whether the existing plan survives the event. If it does, the plan is reused and the model is not called at all — many operational events do not invalidate the current route.

When a candidate fails validation, the specific violations are serialized back to the model as a structured correction prompt, for up to three attempts, before the workflow falls through to the solver.

7 The Validation Contract

7.1 Rules

Every candidate plan, regardless of origin, is checked against the same deterministic rules:

- every active required customer is served exactly once;
- inactive or unavailable customers are not served;
- no unknown customer or vehicle identifiers appear;
- no blocked arc is traversed;
- vehicle capacity is respected;
- every route starts and ends at the depot.

Two properties make this more than a checklist. First, the validator recomputes load and distance from the scenario rather than trusting the values in the candidate; the model supplies only the visit sequence, so a hallucinated number cannot reach the output. Second, validation failure is treated as a prompt rather than an error, feeding the violations back for correction.

7.2 Repair and Fallback

Repair is deliberately conservative. It performs only structural cleanups that cannot invent new optimization decisions: scenario identifier alignment, removal of inactive or duplicated stops, depot boundary normalization, and recomputation of load and distance. Capacity violations, missing customers and any genuine rerouting remain the solver's responsibility.

The repaired plan is re-validated before acceptance. If it still fails, the solver fallback produces an authoritative plan, which is itself validated before being returned.

7.3 Traceability

Every node appends trace metadata, exposed in the frontend as an agent execution panel: extraction confidence and method, candidate source, validation status, repair and fallback decisions, and the final route source. This is what makes the central claim inspectable rather than asserted — an operator can see which component produced the plan they are looking at.

8 AI Engineering Components

Beyond the routing policy and the solver, the system carries three components that make the conversational surface work: a rolling conversation memory, a retrieval layer over project documentation, and a route question-answering pipeline. They are described here because they constitute a meaningful part of the implementation and are visible in the operator interface.

8.1 Conversation Memory and Context Window

Conversations, messages and agent runs are persisted in MongoDB. On top of the raw message log, the system maintains a *context window* per conversation — a compact, bounded summary of state that survives across turns without replaying the full history into the prompt.

Field	Content
`summary`	Rolling narrative: latest event, validation, replanning impact.
`recent_message_ids`	Identifiers of the last N messages.
`facts`	Durable facts accumulated for the conversation.
`open_constraints`	Active operational constraints, capped at the last 20.
`last_event`	Most recent structured event.
`last_plan`	Most recent accepted plan.

The window is rebuilt after each agentic run. The previous summary is carried forward and extended with the new event, the validation outcome, and the replanning impact — distance delta and removed customers — then truncated to a character budget, keeping the most recent text. Open constraints accumulate without duplication so that an active blockage remains in scope across turns.

Two details matter operationally. First, when a turn is answered as a conversational follow-up rather than a replan, the window does not re-record an event, which prevents a question about an existing disruption from being mistaken for a new one. Second, because the window carries `last_plan`, the system can answer questions about the current route without re-solving.

8.2 Retrieval-Augmented Generation

The retrieval layer indexes project documentation so that questions about system behaviour can be answered from written sources rather than model recall.

Ingestion and chunking. Ingestion accepts `.md`, `.txt`, `.tex`, `.py`, `.json`, `.yaml`, `.yml` and `.toml`, skipping build and environment directories. Text is chunked at 1200 characters with 160 characters of overlap. Chunk boundaries snap to the nearest newline, or failing that the nearest space, but only when that boundary falls past the midpoint of the window — this avoids degenerate short chunks while keeping splits off mid-word.

Embeddings. Two interchangeable backends implement a common `EmbeddingClient` protocol:

- `hash` (default): a deterministic local embedding. Each token is hashed with BLAKE2b into a bucket of a 384-dimensional vector with a signed contribution, and the result is L2-normalized. It requires no model server, which makes the retrieval pipeline testable on CPU and reproducible across runs.
- `api`: an OpenAI-compatible `/embeddings` endpoint, with returned vectors fitted to the configured dimension by truncation or zero-padding.

Storage and search. A repository protocol has two implementations. The in-memory version scores chunks by cosine similarity in Python and is used for tests. The production version stores chunks in PostgreSQL with the `pgvector` extension, creating the extension and schema on first use, with the embedding column typed `vector(d)` at the configured dimension. Retrieval uses the cosine distance operator:

```
SELECT ..., 1 - (c.embedding <=> %s::vector) AS score
FROM chunks c
ORDER BY c.embedding <=> %s::vector
LIMIT %s
```

The subtraction converts distance into a similarity score for ranking and display.

8.3 Route Question Answering

Not every driver message is a disruption. Questions such as "what is my next stop?" or "why did the plan change?" should be answered without mutating the scenario. These are handled by a separate pipeline in the conversation service, whose stages are surfaced individually in the agent execution panel:

1. `route_lookup` — resolve the route identifier to a stored route, driver and scenario;
2. `route_fact_builder` — assemble deterministic facts from the stored plan: stop sequence, distances, loads, status;
3. `route_qa_rag` — retrieve supporting documentation chunks when retrieval is enabled;
4. `route_qa_model` — compose the natural-language answer with the orchestrator model.

The ordering is deliberate. Route facts are computed from persisted data before the model is called, so the answer is grounded in the actual plan rather than generated from the conversation alone. If the model call fails, the pipeline records the failure in the trace and degrades to a fact-based answer rather than returning nothing.

The classification between question answering and replanning matters: a message implying a route change must be routed into the replanning workflow, where the validation contract applies. The Q&A path is explicitly not an authority on feasibility.

9 Dataset Generation

9.1 Methodology

Datasets were generated synthetically for scale, repeatability and reliable labels:

1. generate a CVRP instance with depot, customers, demands, vehicles, capacities and distance matrix;
2. solve the base plan with Pyomo + HiGHS;
3. apply an operational event;
4. re-solve the disrupted scenario;
5. validate the replanned route;
6. serialize scenario, event and target into compact SFT JSONL;
7. convert compact rows into chat-message JSONL for instruction fine-tuning.

The solver therefore labels its own training set. The model learns to imitate prescriptive decisions while the same engine remains available as a deterministic oracle at runtime.

9.2 Generated Datasets

Dataset	Total	Train	Validation	Test	Purpose
`training_compact_30k`	30000	24000	3000	3000	Initial compact curriculum.
`training_capacity_tight_50k_parallel`	50000	40000	5000	5000	Emphasis on capacity pressure.
`training_blocked_arc_20k_parallel`	20000	16000	2000	2000	Emphasis on blocked arcs.
`training_curriculum_100k`	100000	80000	10000	10000	Consolidated curriculum dataset.
`training_error_driven_20k`	20000	16000	2000	2000	Continuation focused on observed errors.
`training_error_driven_10k_v6`	10000	8000	1000	1000	Additional continuation after v5.

The split strategy was 80% training, 10% validation, 10% test. Audits found zero invalid rows in the principal training datasets.

10 Fine-Tuning Strategy

10.1 Training Algorithm

Supervised fine-tuning with LoRA/QLoRA, using Hugging Face Transformers for model loading, TRL `SFTTrainer` for supervised fine-tuning, PEFT for adapter management and bitsandbytes for 4-bit loading. The optimization target is causal language-model loss over the assistant response tokens.

10.2 Final LoRA Configuration

Parameter	Value
Base model	`Qwen/Qwen2.5-7B-Instruct`
Method	LoRA / QLoRA-style loading
PEFT type	`LORA`
Task type	`CAUSAL_LM`
Rank r	16
`lora_alpha`	32
`lora_dropout`	0.05
Bias	`none`
Target modules	`q_proj,k_proj,v_proj,o_proj,gate_proj,up_proj,down_proj`
Quantization	4-bit NF4 with double quantization
Compute dtype	bfloat16

11 Training Methodology and Parameters

Training was conducted in stages, each continuation resuming the previous adapter and evaluated against the same fixed benchmark before deciding whether to continue or stop.

Version	Train rows	Eval rows	Steps	Batch	LR	Time	Notes
v3	24000	500	3000	1×8 accum.	$5 \cdot 10^{-5}$	5h53m	First stable full training on the 30K compact dataset.
v4	56000	1000	2300	8	$1.7 \cdot 10^{-5}$	3h44m	Hard-data continuation using SDPA and checkpoint resume.
v5	16000	1000	1500	8	$1 \cdot 10^{-5}$	2h22m	Error-driven continuation. Selected version.
v6	8000	500	800	8	$7 \cdot 10^{-6}$	1h18m	Additional continuation; not selected.

Total training time across the four runs was approximately 13 hours on a single RTX 3090. The decreasing learning rate across stages reflects a deliberate curriculum: broad fitting first, then progressively finer correction against observed failure modes.

The stabilized setup used `--max-seq-length2048`, `--bf16`, `--bnb-compute-dtypebfloat16`, `--optimadamw_torch`, `--lr-scheduler-typecosine`, `--max-grad-norm0.3`, `--no-eval-during-train`, `dataset_num_proc=1` and `dataloader_num_workers=0`. One epoch per run.

The representative v5 command was:

```
uv run python scripts/train_lora.py \
  --model-id Qwen/Qwen2.5-7B-Instruct \
  --adapter-path outputs/models/adaptiveroute-qwen2_5-7b-lora-hard70k-v4-b8-sdpa-resume700 \
  --dataset-dir data/chat_training_error_driven_20k \
  --output-dir outputs/models/adaptiveroute-qwen2_5-7b-lora-error20k-v5 \
  --max-seq-length 2048 \
  --epochs 1 \
  --max-steps 1500 \
  --learning-rate 1e-5 \
  --warmup-steps 50 \
  --max-grad-norm 0.3 \
  --lr-scheduler-type cosine \
  --per-device-batch-size 8 \
  --gradient-accumulation-steps 1 \
  --train-limit 16000 \
  --eval-limit 1000 \
  --bf16 \
  --bnb-compute-dtype bfloat16 \
  --optim adamw_torch \
  --save-steps 100 \
  --logging-steps 10 \
  --no-eval-during-train
```

12 Evaluation

12.1 Training Metrics

Loss converged to the 0.26–0.28 range after the initial learning stage, and mean token accuracy stabilized around 0.89. These metrics indicate that the model learned the response format and common routing patterns, but they are not evidence of operational correctness: the curve is essentially flat across v4, v5 and v6 while the operational metrics still differ. This is precisely why loss was not used as the selection criterion.

Version	Train loss	Final token accuracy	Tokens processed	Runtime
v3	0.2796	0.8922	$1.614 \cdot 10^7$	21190s
v4	0.2659	0.8915	$1.241 \cdot 10^7$	13470s
v5	0.2654	0.8911	$8.081 \cdot 10^6$	8510s
v6	0.2653	0.8917	$4.307 \cdot 10^6$	4700s

12.2 Operational Evaluation

All versions were evaluated on the same 1,000-example sample from `data/training_curriculum_100k/sft_test.jsonl`. The primary metric is the feasible-candidate rate. Exact match was tracked but treated as secondary, because a CVRP instance commonly admits several equally optimal plans, so matching the reference sequence measures conformity rather than quality.

Model	Valid JSON	Feasible	Exact match	Capacity violations	Blocked-arc violations
v3	100.0%	91.7%	4.6%	72	11
v4	100.0%	92.9%	5.8%	61	10
v5	100.0%	94.4%	5.6%	46	9
v6	100.0%	94.0%	5.4%	50	10

The improvement from v3 to v5 is substantial: feasibility rises 2.7 points and capacity violations fall by 36%. The comparison between v5 and v6 requires more care. A difference of 0.4 percentage points on $n = 1000$ corresponds to four examples; the 95% confidence interval for a proportion of 0.944 at that sample size is approximately ± 1.4 points, so v5 and v6 are statistically indistinguishable on feasibility. The honest statement is that the additional continuation produced no measurable improvement, and v5 was retained as the stopping point rather than v6 being shown to regress.

v5 was selected because it holds the best observed feasibility, the lowest count in both violation categories, and because the curve had flattened — continuing to train without a new measured error target was not justified.

12.3 Capacity Boundary

Feasibility on the benchmark describes performance at the size the model was trained for. A separate benchmark measured how performance degrades as scenario cardinality grows, using strict validation and a viability threshold of 90%.

Customers	Strict valid rate	Avg. candidate latency	Dominant failure mode
8	100%	1.59s	—
9	100%	1.71s	—
10	60%	3.99s	customer omitted
12	33%	—	customer omitted
14	0%	—	customer omitted

The usable boundary is 8–9 customers. Above it, the model omits customers rather than routing them poorly, and the omissions concentrate on identifiers at or beyond the edge of the training distribution, such as `C10` and above. This is a cardinality generalization limit rather than a routing-quality problem, and it will not be fixed by prompt engineering; it requires training data at larger cardinalities.

Two caveats apply. The samples are small — three to five scenarios per size — so the intermediate rates carry wide uncertainty, though the trend and the qualitative failure mode are consistent. Second, exceeding the boundary does not produce incorrect plans in production: the cascade falls through to the solver on every request, trading latency for the same guarantee.

13 Engineering Practices

The system comprises roughly 11,800 lines of Python across 73 modules, with 24 test files. All domain and record types are immutable frozen dataclasses. Services and repositories are separated, with in-memory repository implementations as the default, so the entire test suite runs without a GPU, a database or a model server.

Continuous integration is split into two jobs: fast API and agentic tests, and solver authority tests executed in forked subprocesses. The split is necessary because the Pyomo/HiGHS stack can trigger a native segmentation fault after repeated solver invocations within one Python process.

Driver credentials are stored as bcrypt hashes and authentication issues signed JWTs; CORS origins are configurable rather than open; requests emit structured JSON logs with correlation identifiers. A placeholder JWT secret is detected at startup and replaced with a process-local random value, so a deployment that omits configuration cannot sign tokens with a publicly known key.

14 Limitations

14.1 Model Limitations

- The model does not guarantee feasibility or optimality. On the 1,000-example benchmark, v5 still produced 46 capacity violations and 9 blocked-arc violations.
- Exact match is low, though this is partly expected for CVRP.
- Training data is entirely synthetic. No real traffic, fleet telemetry, driver constraints, service times or geospatial road-network data were used for training.
- The usable scenario size is 8–9 customers, as measured in Section 12.3.

14.2 Architectural Limitations

- The MTZ formulation has a loose linear relaxation; larger fleets would benefit from a stronger branch-and-cut formulation or a dedicated routing solver.
- Solver calls still execute inside the request lifecycle. Production use requires a worker queue so that optimization leaves the request path.
- Validation proves that a plan is correct *for the mutated scenario.* It cannot prove that the mutation corresponds to what the driver actually said. Event extraction is the one stage the trust boundary does not cover, and a misread message produces a perfectly valid answer to the wrong question.

- The default embedding backend is the deterministic hash embedding. It makes the retrieval pipeline reproducible and testable without a model server, but it is a bag-of-tokens projection with no semantic representation: retrieval quality under it is lexical at best. Meaningful retrieval requires configuring the API embedding backend, and the current deployment has not been evaluated for retrieval precision or recall.
- List endpoints support offset pagination but not cursor pagination or server-side filtering.

14.3 Infrastructure Limitations

- Long-running training and generation jobs exposed native failures, including `exitstatus139`, double-free errors, and a `torch.nn.Module.train` runtime issue.
- FlashAttention could not be installed: the local environment used Python 3.12 with a Torch/CUDA 13.0 stack for which no compatible prebuilt wheel was available. The stable workaround was SDPA, reduced preprocessing parallelism, evaluation disabled during training, and `adamw_torch`.

15 Refinement Strategy

In priority order:

1. harden event extraction with confidence thresholds and explicit confirmation before mutating a scenario, closing the gap identified in Section 14;
2. extend the cardinality boundary with curriculum data at 10, 12, 16 and 24 customers, and evaluate by scenario size rather than a random split;
3. move optimization off the request path with a worker queue, persisted solver artifacts and time limits tuned per scenario class;
4. instrument the cascade in production, tracking fallback rate and violation modes as the live measure of policy quality;
5. generate new error-driven datasets from actual validator failures, and train continuations only when they beat the incumbent on the fixed benchmark;
6. evaluate best-of-N candidate generation with deterministic selection;
7. expand scenario richness: time windows, multiple depots, asymmetric costs, more disruption types;
8. validate against public VRP benchmarks and, ultimately, historical operational routes.

16 Runtime Contract

The selected adapter is `outputs/models/adaptiveroute-qwen2_5-7b-lora-error20k-v5`, with the stable operational alias `outputs/models/adaptiveroute-routing-policy-current`.

```
driver message
-> event extraction        (orchestrator, with deterministic fallback)
-> base solve              (Pyomo + HiGHS)
-> scenario mutation
-> candidate generation    (LoRA v5, ~1.7s)
-> deterministic validation
```

```
-> conservative repair     if invalid
-> Pyomo + HiGHS fallback  if still invalid
-> validated response
```

The accepted response carries the candidate source, the validation result, any rejected violations, route distance and vehicle loads, and the final route JSON.

17 Reproducibility

The results in this report are reproducible from the repository. This section states the entry points; operational detail lives in `README.md` and `docs/DEPLOYMENT.md`.

Running the system. The default Docker profile requires no GPU and no model weights. It starts the API, the frontend, MongoDB and PostgreSQL:

```
cp .env.docker.example .env.docker
./scripts/docker_up.sh
./scripts/smoke_docker_stack.sh
```

Road distances and road-snapped geometry require the OSRM profile, which is opt-in and needs its graph prepared once:

```
./scripts/prepare_osrm_nyc.sh
COMPOSE_PROFILES=routing ADAPTIVEROUTE_MAP_ROUTER_BACKEND=osrm ./scripts/docker_up.sh
```

Without it the system falls back to haversine distances and straight-line geometry. The fallback is silent by design, so the map legend is the reliable indicator of which backend is active.

Tests. The full suite runs against in-memory repositories and requires no GPU, database or model server:

```
uv run pytest
```

Continuous integration splits this into a fast API and agentic job, and a solver job executed with `pytest--forked`, because the Pyomo/HiGHS stack can segfault after repeated solver invocations within one process.

Reproducing the operational evaluation. The feasibility figures in Section 11 are produced in two steps, against the fixed test split:

```
uv run python scripts/generate_model_predictions.py \
  --model-path outputs/models/adaptiveroute-qwen2_5-7b-lora-error20k-v5 \
  --dataset data/training_curriculum_100k/sft_test.jsonl \
  --out outputs/predictions/v5_test.jsonl \
  --limit 1000

uv run python scripts/evaluate_predictions.py \
  --dataset data/training_curriculum_100k/sft_test.jsonl \
  --predictions outputs/predictions/v5_test.jsonl \
  --details-out outputs/predictions/v5_eval_details.jsonl
```

Prediction generation took 20–25 minutes per model on the reference hardware.

Reproducing the capacity benchmark. The boundary in Section 11.3 is produced by sweeping scenario size against the served policy:

```
uv run python scripts/benchmark_routing_policy_capacity.py \
  --customer-grid 8,9,10,12,14 \
  --samples-per-size 5 \
  --profile balanced \
  --event-types block_arc,customer_unavailable \
  --viability-threshold 0.9 \
  --out outputs/evaluations/routing_policy_capacity.json
```

Vehicle count defaults to $\lceil \text{customers}/8 \rceil$ and the seed sequence starts at 10,000, so runs are repeatable. Datasets are regenerated deterministically from a seed and are intentionally excluded from version control.

18 Training and Test Environment

Item	Observed value
Operating system	Ubuntu 24.04.4 LTS
Kernel	Linux 6.8.0-138-generic
CPU	Intel Core i9-14900K, 32 logical CPUs
GPU	NVIDIA GeForce RTX 3090
VRAM	24576 MiB
NVIDIA driver	580.173.02
Compute capability	8.6
Python	3.12.3 via `uv`
Torch	2.13.0, CUDA 13.0
Transformers	5.15.1
TRL	1.10.0
PEFT	0.20.0
bitsandbytes	0.50.1
datasets	5.0.1
Pyomo	6.10.1
HiGHS/highspy	1.15.1

19 Conclusion

AdaptiveRoute demonstrates that an unreliable fast component can be used safely when the architecture never depends on it being correct. The fine-tuned routing policy returns feasible replans 94.4% of the time at the scenario size it was trained for; the remaining 5.6% is absorbed by validation, conservative repair and an exact solver fallback, so the failure rate is a latency cost rather than a correctness risk.

Three results are worth separating. The first is the policy itself, which is a competent but bounded artifact: useful to 8–9 customers, measured rather than assumed, with a clear path to extension. The second is the two-model split, which assigns breadth and precision to different components rather than asking one model to provide both. The third, and the most durable, is the validation contract: a single deterministic arbiter that every plan must pass regardless of origin, with the trace exposed so that an operator can see which component produced what.

The remaining open edge is event extraction. Validation proves a plan is correct for the scenario it was given; it cannot prove the scenario matches what the driver reported. Closing that gap is the next substantive piece of work, and it is a guardrail problem rather than a modelling one.

References

[1] G. B. Dantzig and J. H. Ramser. The Truck Dispatching Problem. *Management Science*, 6(1):80–91, 1959. https://doi.org/10.1287/mnsc.6.1.80

[2] C. E. Miller, A. W. Tucker and R. A. Zemlin. Integer Programming Formulation of Traveling Salesman Problems. *Journal of the ACM*, 7(4):326–329, 1960. https://doi.org/10.1145/321043.321046

[3] O. Vinyals, M. Fortunato and N. Jaitly. Pointer Networks. *Advances in Neural Information Processing Systems*, 2015. https://arxiv.org/abs/1506.03134

[4] W. Kool, H. van Hoof and M. Welling. Attention, Learn to Solve Routing Problems! *International Conference on Learning Representations*, 2019. https://arxiv.org/abs/1803.08475

[5] Y. Bengio, A. Lodi and A. Prouvost. Machine Learning for Combinatorial Optimization: A Methodological Tour d'Horizon. *European Journal of Operational Research*, 290(2):405–421, 2021. https://arxiv.org/abs/1811.06128

[6] A. Vaswani et al. Attention Is All You Need. arXiv:1706.03762, 2017. https://arxiv.org/abs/1706.03762

[7] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021. https://arxiv.org/abs/2106.09685

[8] T. Dettmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314, 2023. https://arxiv.org/abs/2305.14314

[9] Qwen Team. Qwen2.5 Technical Report and Qwen2.5-7B-Instruct model card. https://arxiv.org/abs/2412.15115 and https://huggingface.co/Qwen/Qwen2.5-7B-Instruct

[10] M. L. Bynum et al. Pyomo – Optimization Modeling in Python. Third Edition, Springer, 2021. https://www.pyomo.org/citing-pyomo

[11] HiGHS. High-performance open-source software for linear optimization. https://highs.dev/

[12] D. Luxen and C. Vetter. Real-time Routing with OpenStreetMap Data. *ACM SIGSPATIAL GIS*, 513–516, 2011. https://doi.org/10.1145/2093973.2094062